DATA STRUCTURE FUNDAMENTALS

Prof. Dr. Md. Rafiqul Islam

CHAPTER 7

TREE

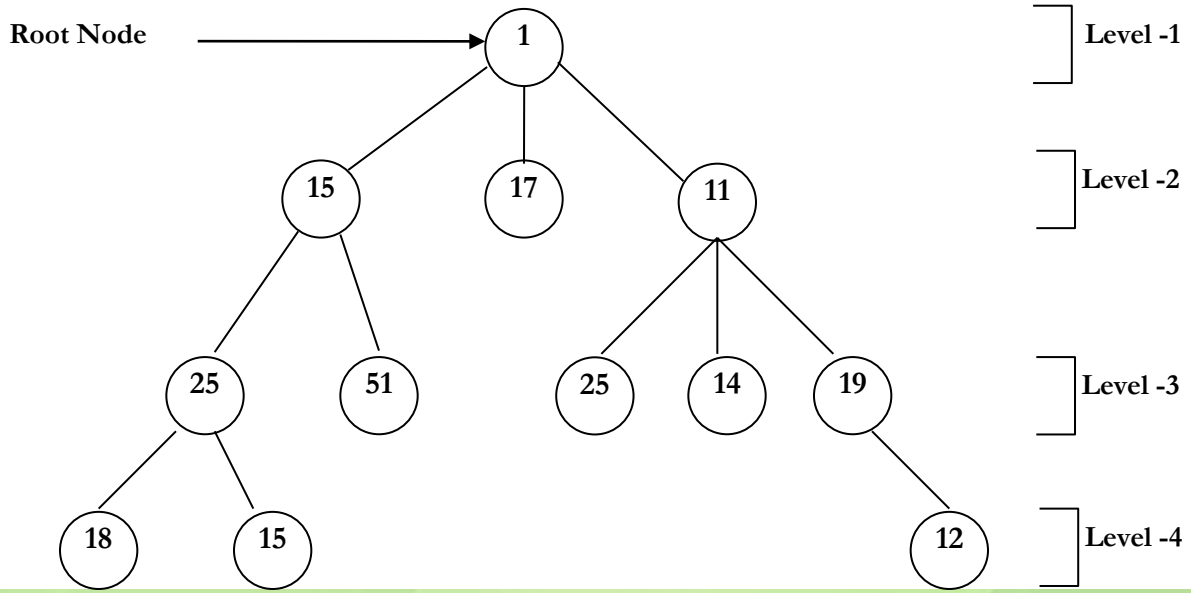
TREE

- We see tree in nature. The tree has root, branches, sub-branches and leaves. From the concept of natural tree, the computer scientists get the idea of a data structure, which is graphically similar to natural tree.
- Natural tree is a bottom-up figure. However, the graphical representation of the data structure tree is a top-down figure.

TREE

- A tree is a finite collection of nodes that has one to many relationship among nodes.
- A tree is a hierarchical structure.
- An ordered tree is a list of nodes that has a specially designated node called root node.
- The connection line between two nodes is called edge.
- The node that has no child node is called leaf node. The node that has child node is called parent node.
- A tree can be implemented (stored in memory) as an array or a linked list.

TREE



- Figure 7.1: A Tree

Binary Tree

- A binary tree is finite set of nodes, where there is a special node called root node and every node (including root node) has at best two children.

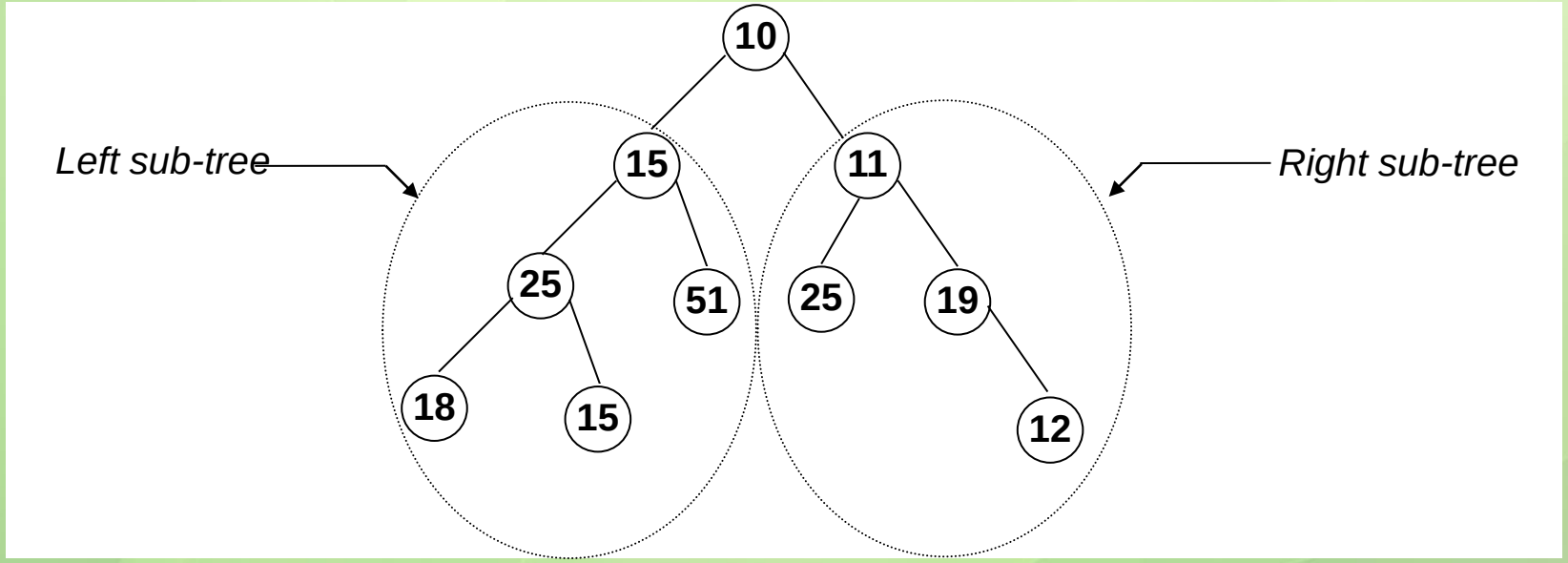
- The trees excluding root node are called left sub-tree and right sub-tree.

Binary Tree

- Each node can have at most 2 children



Binary Tree

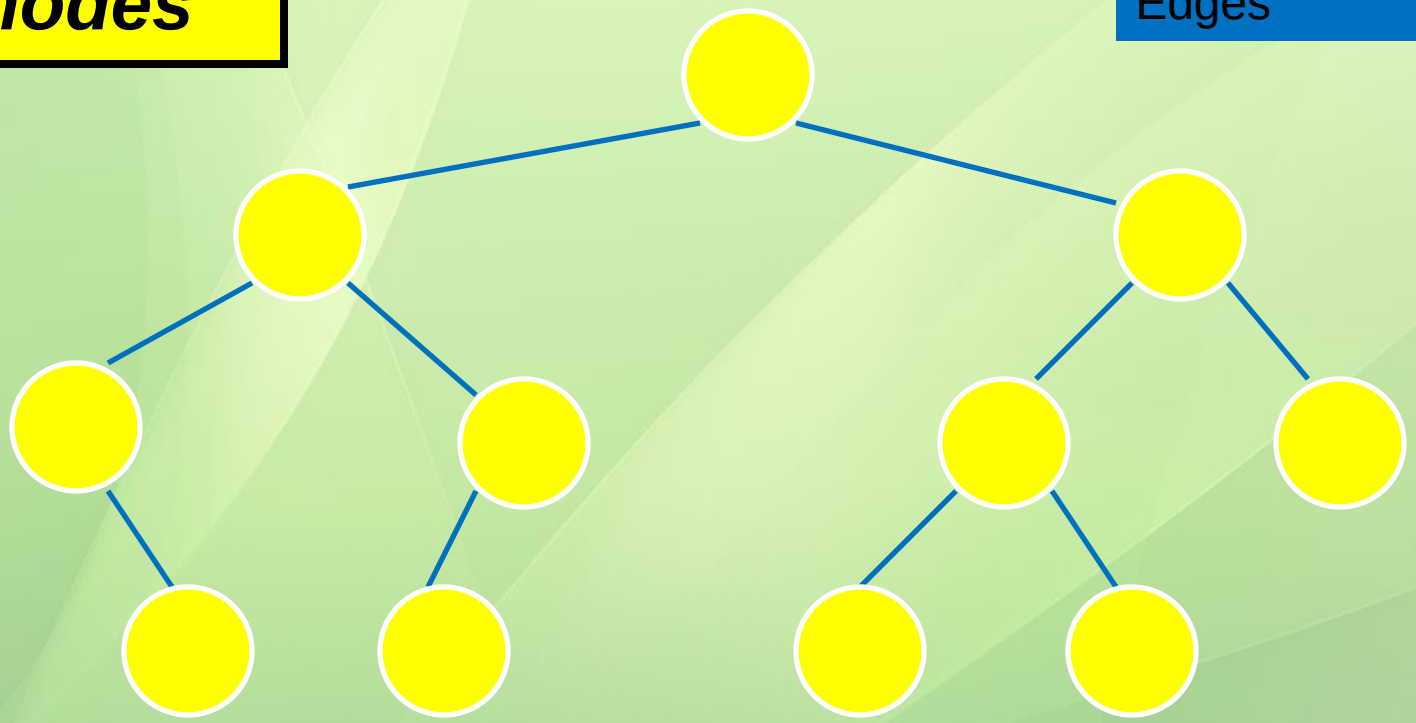


- Figure 7.2: A binary tree

Parts of a Tree

nodes

Edges

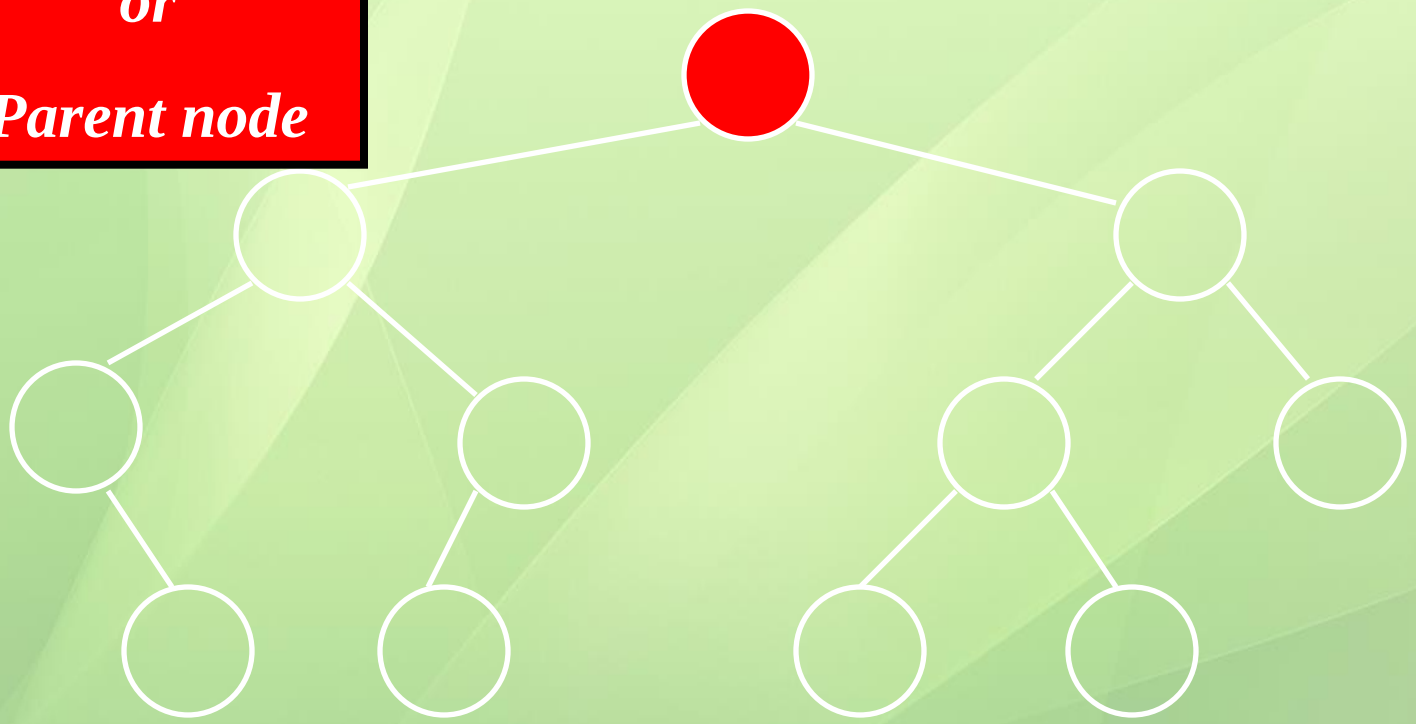


Parts of a Tree

Root

or

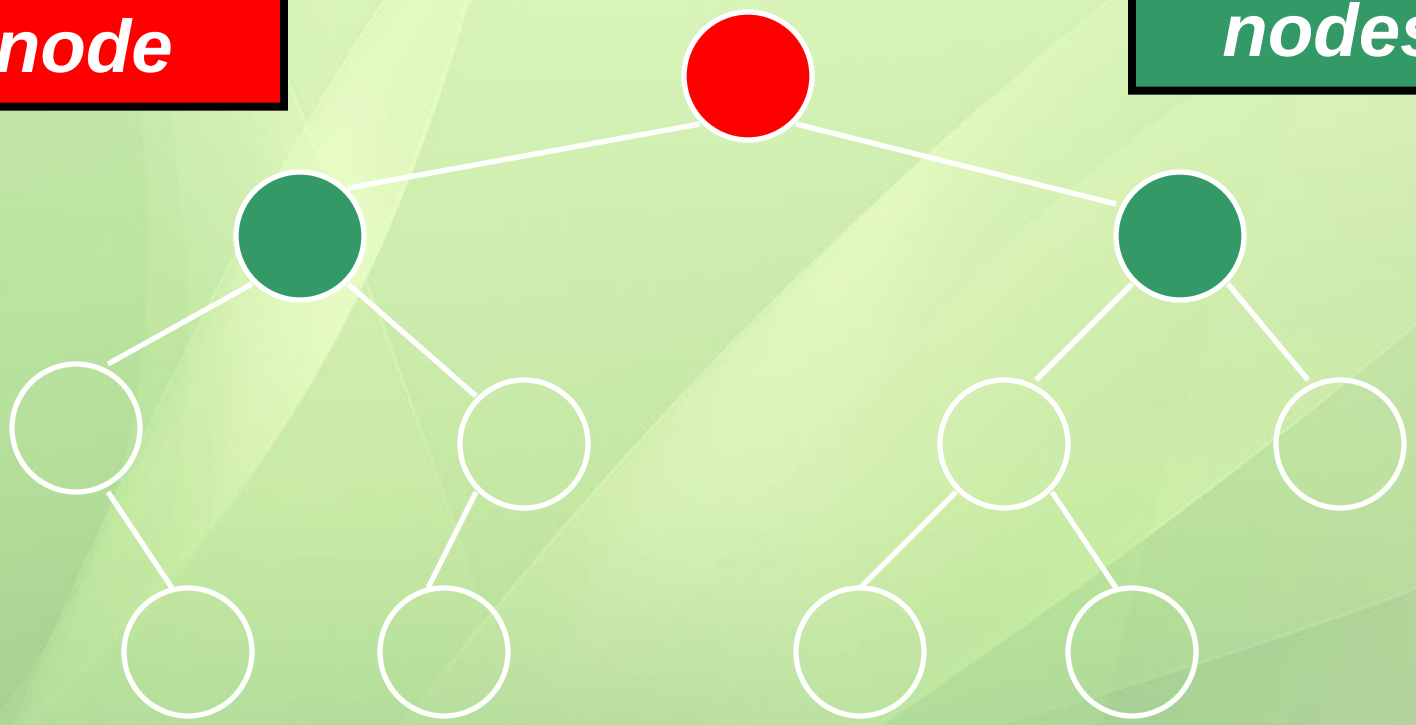
Parent node



Parts of a Tree

parent node

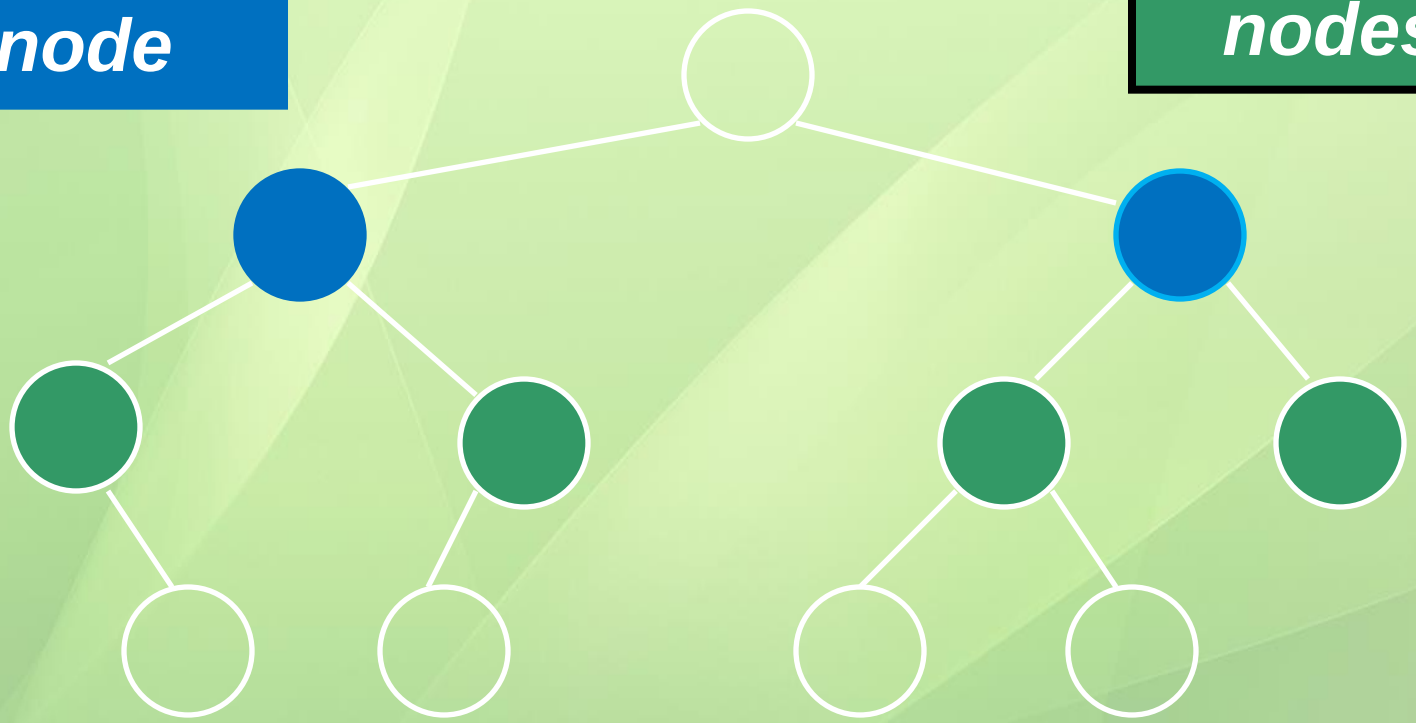
child nodes



Parts of a Tree

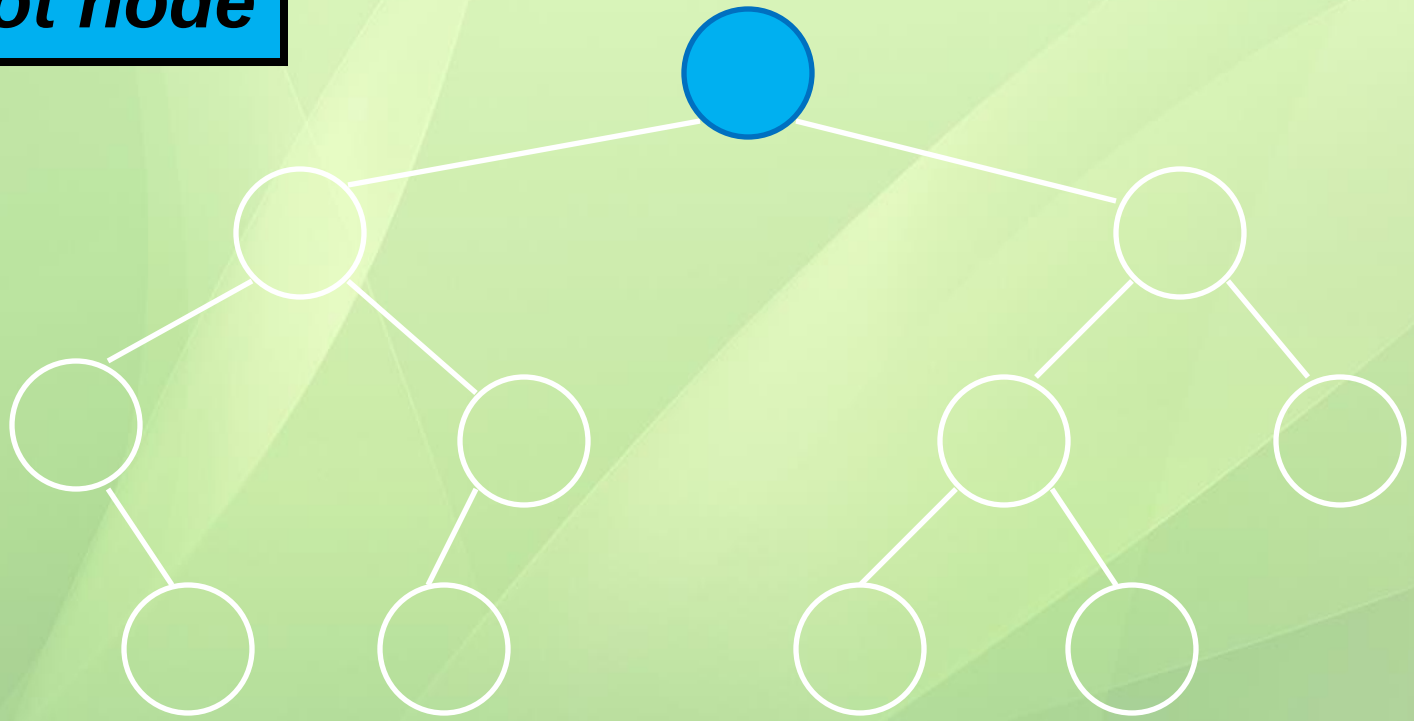
parent node

child nodes



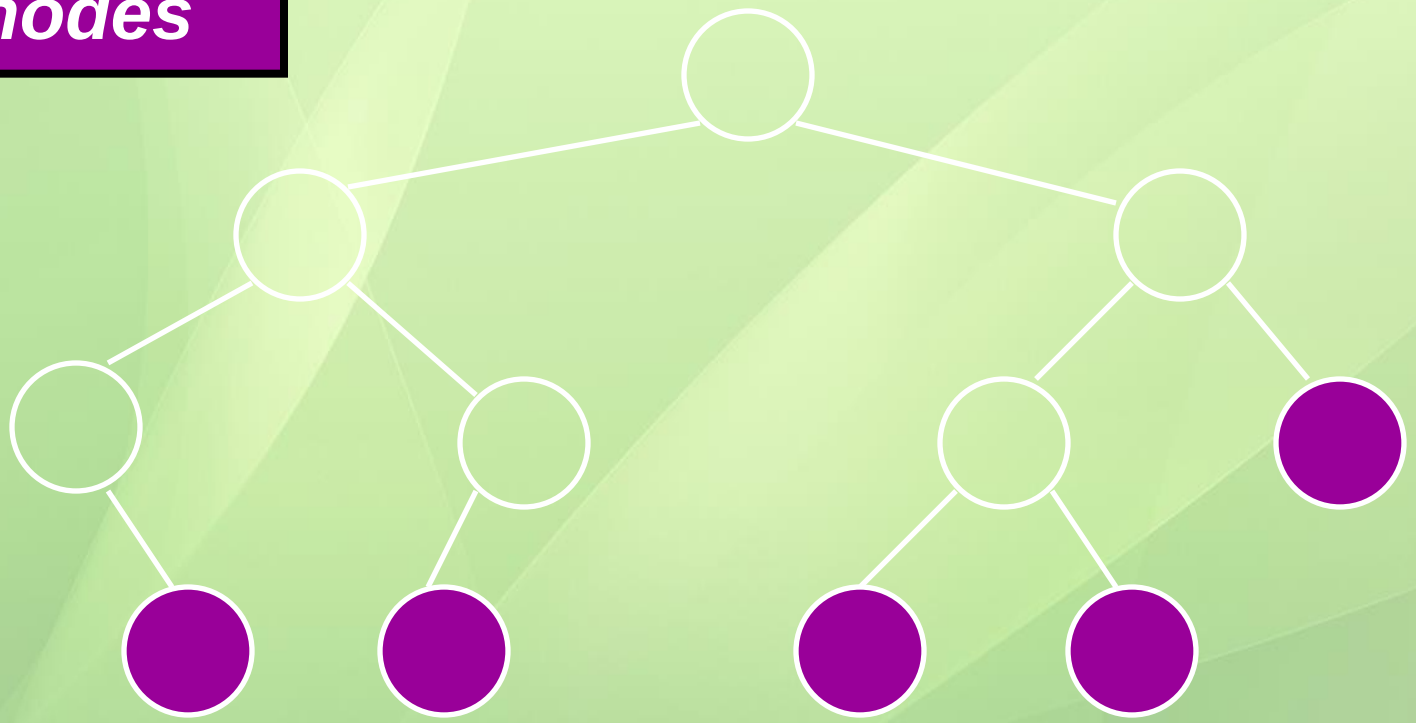
Parts of a Tree

root node



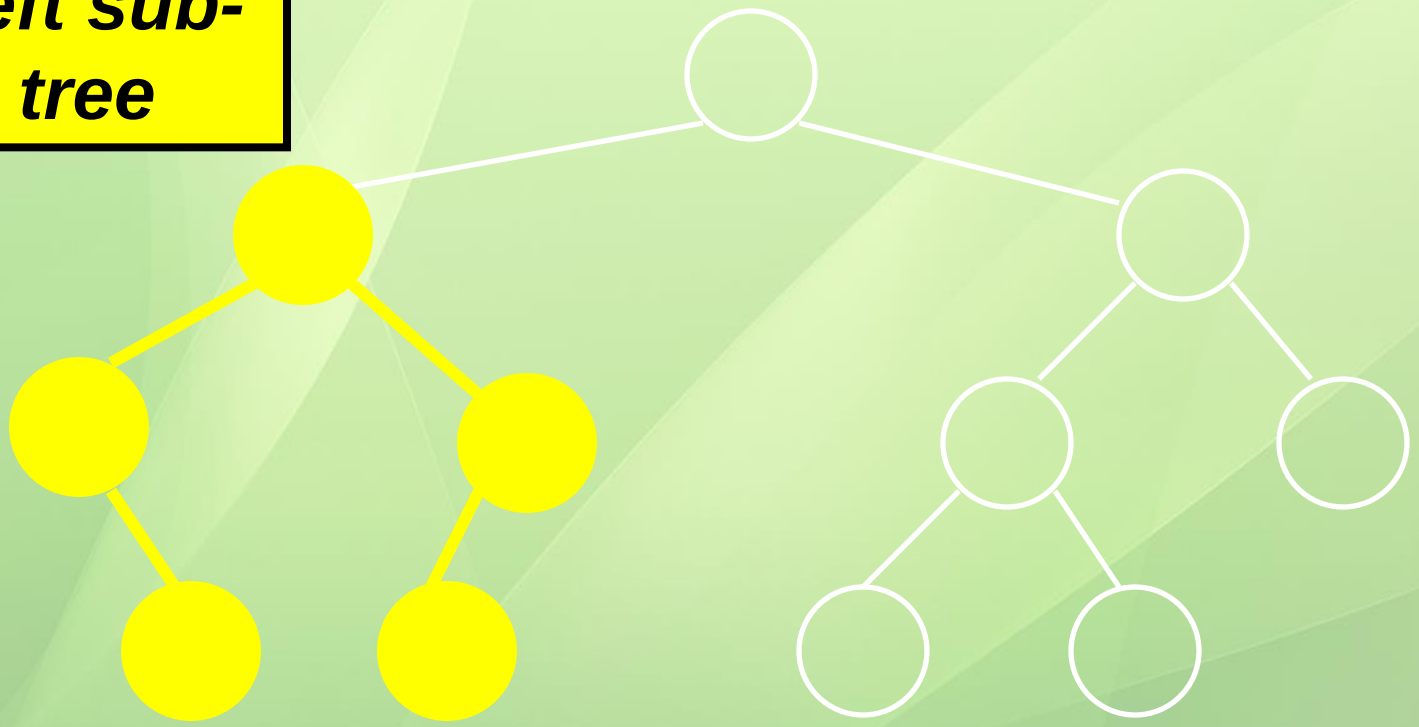
Parts of a Tree

leaf nodes



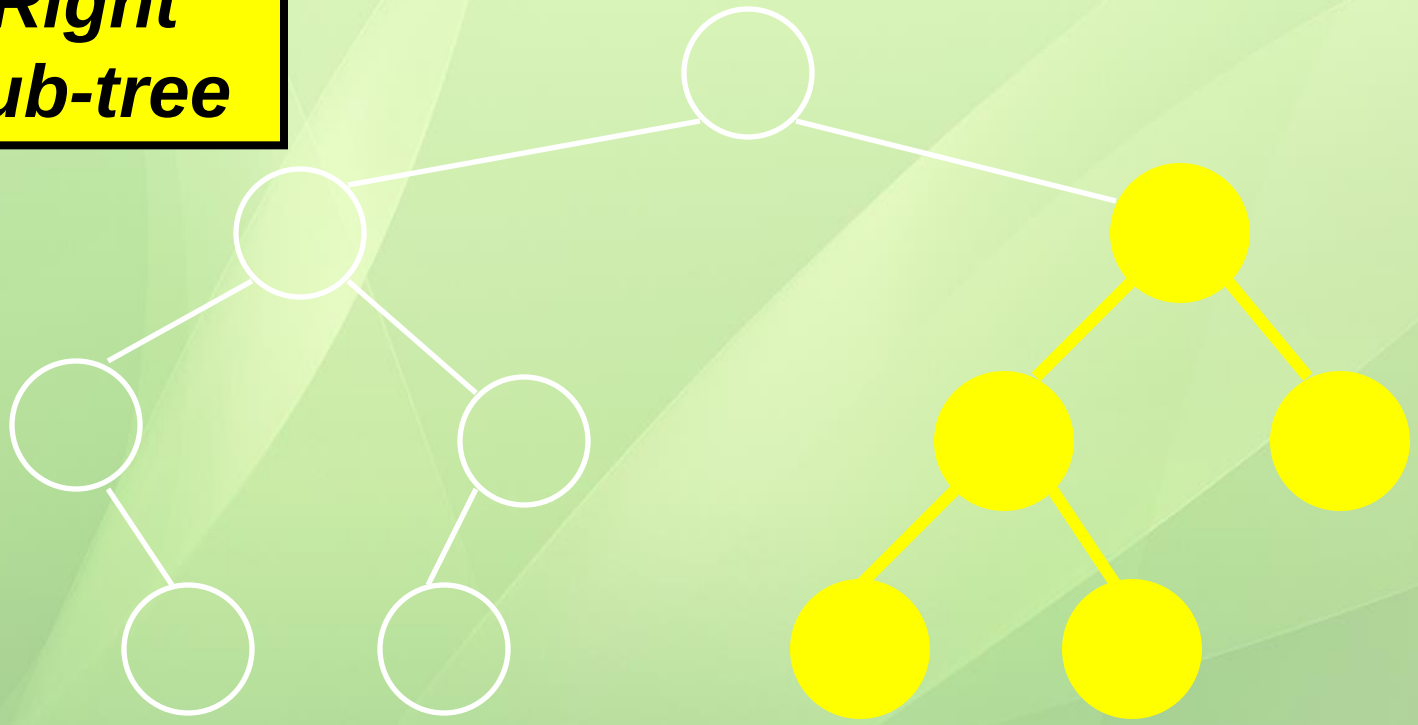
Parts of a Tree

Left sub-tree



Parts of a Tree

Right sub-tree



Parts of a Tree

sub-tree



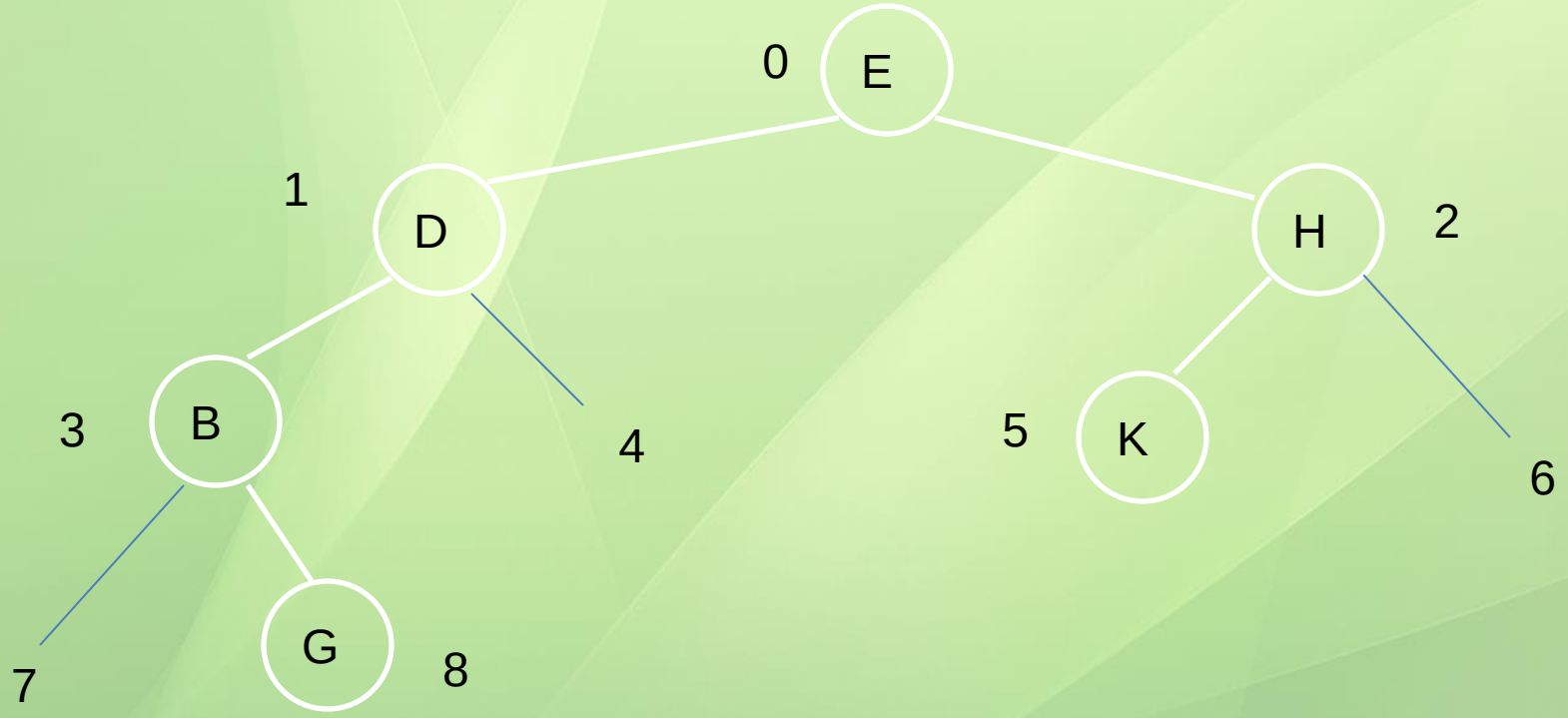
Parts of a Tree

sub-tree



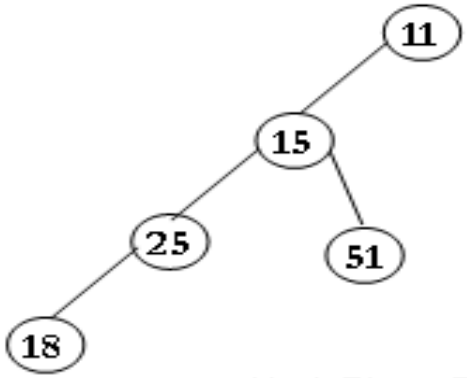
Array Representation of a Tree

0, 1, 2, 3, . . .are positions or indices



0	1	2	3	4	5	6	7	8
E	D	H	B		K			G

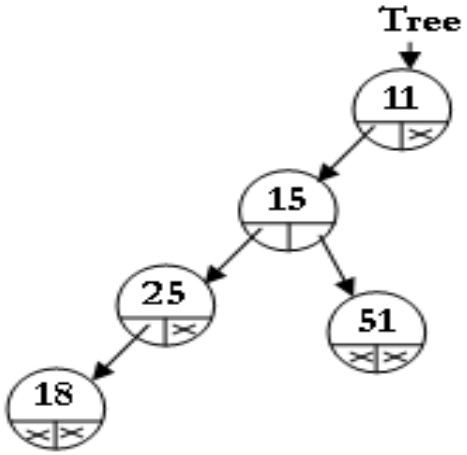
Binary Tree Representation



(a): A Binary Tree

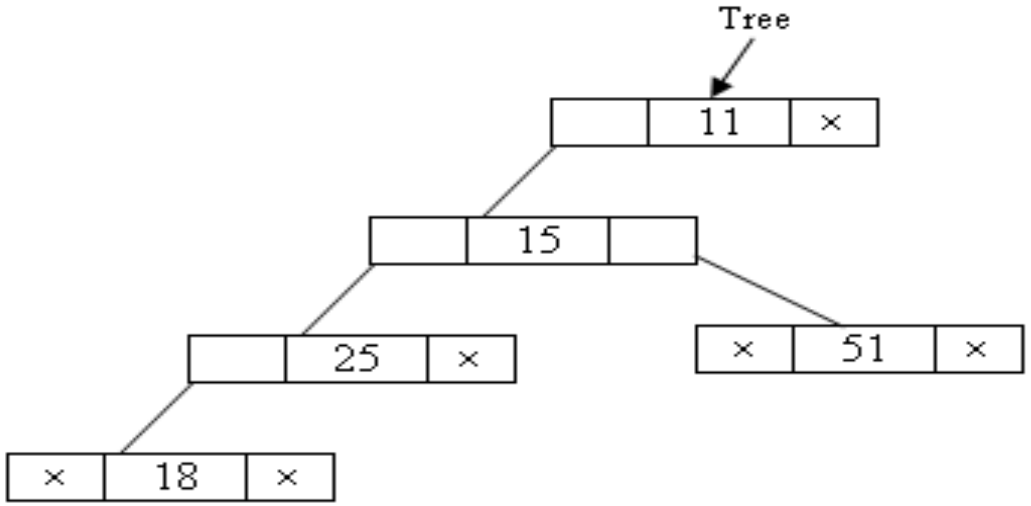
11	15	-	25	51			18

(b): Tree as an array



(c): Tree as a linked list (nodes are shown in round shape)

Binary Tree



(d) Tree as a linked list (nodes are shown in rectangular shape)

Figure 7.3: Tree implementation (store in memory)

Binary Tree

If there are *k* levels in a binary tree, then the maximum number of nodes in the tree is as follows:

$$n = 2^k - 1.$$

For example, if $k = 3, n = 7$

$k = 4, n = 15.$

When n is known, then

$$k = \lceil \log_2 (n+1) \rceil$$ // $\lceil x \rceil$ means ceiling of x to the next integer

For example,

when $n = 17$; $k = \lceil \log_2 (17+1) \rceil = \lceil \log_2 (18) \rceil = \lceil 4.1..... \rceil = 5$

when $n = 34$; $k = \lceil \log_2 (34+1) \rceil = \lceil \log_2 (35) \rceil = \lceil 5.2.... \rceil = 6$; etc.

Binary Tree

- **Full binary tree:**
  - If a binary tree contains nodes in such a way that every node has at least two children except the leaf nodes, then the tree is called a full binary tree.
- **Complete binary tree:**
  - If a binary tree contains nodes in such a way that every level except the deepest has as many nodes as possible and the nodes of the deepest level are in as left as possible, the tree is called complete binary tree.
  - All full binary trees are complete binary trees.

Binary Tree

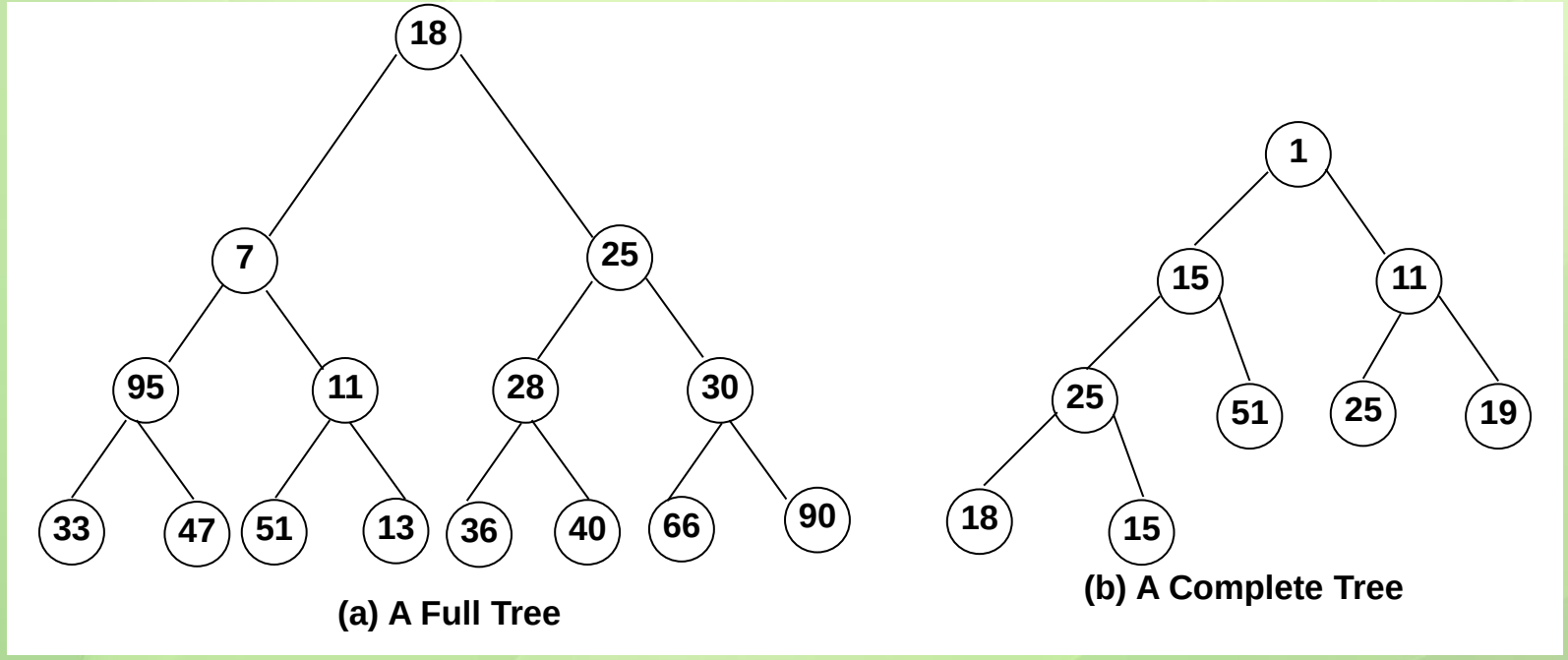


Figure 7.4: Pictorial view of Full tree and complete tree

Binary Tree in Array

Parent child relationship(from parent to child):

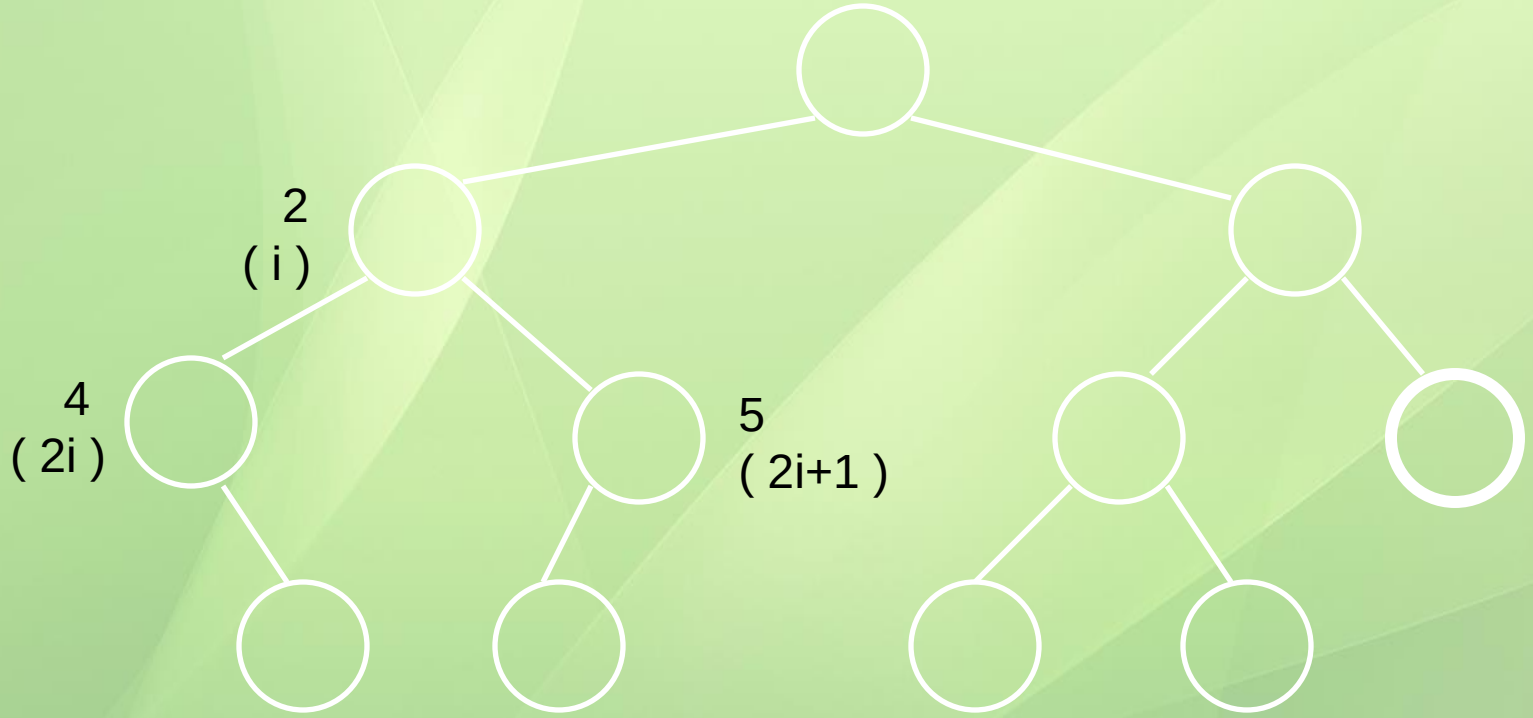
If we consider the root's **position is 1**,

Parent's position(index) = i,

Left child's position(index) = 2i,

Right child's position(index) = 2i + 1.

Parent Child Relationship



Binary Tree in Array

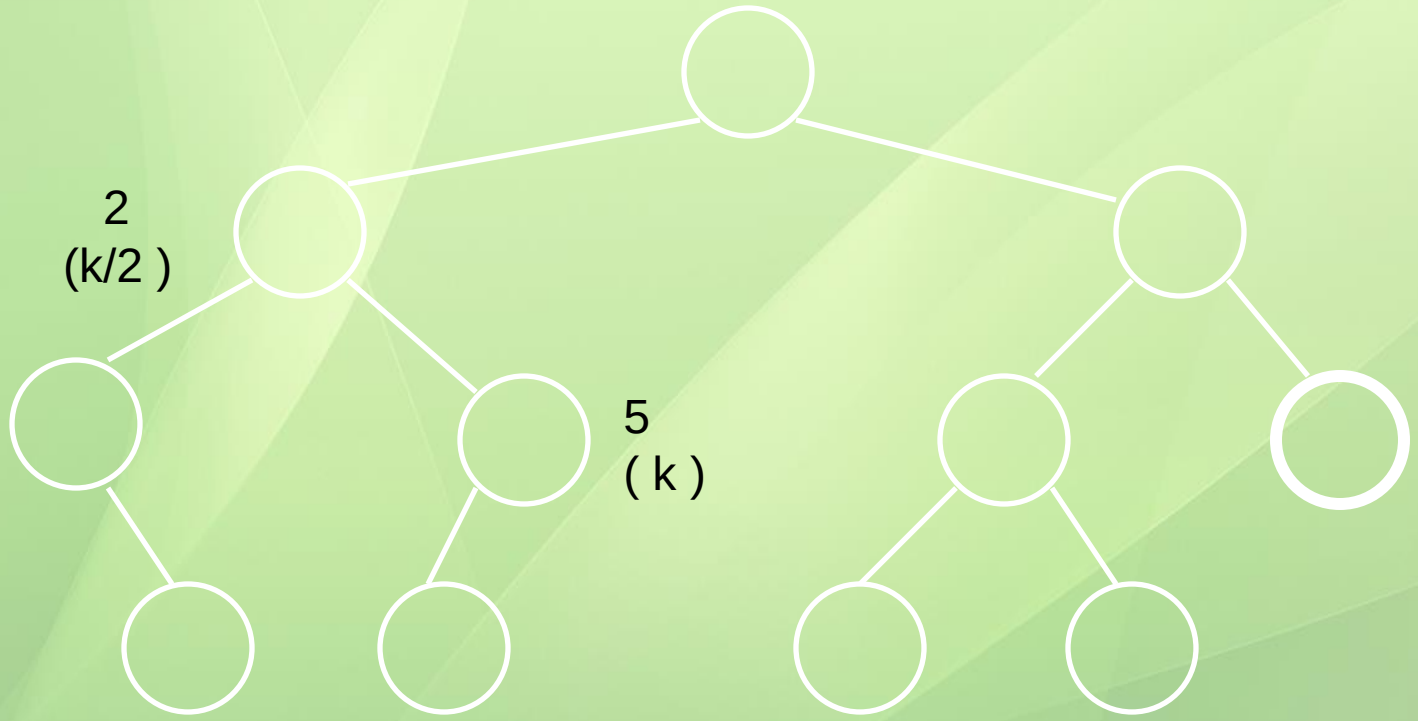
Child parent relationship (from child to parent):

Child's position (index): k

Parent's position: k/2, / denotes Integer division.

If the binary tree is a complete then array representation is efficient.

Child Parent Relationship



Binary Tree in Array

Parent child relationship(from parent to child):

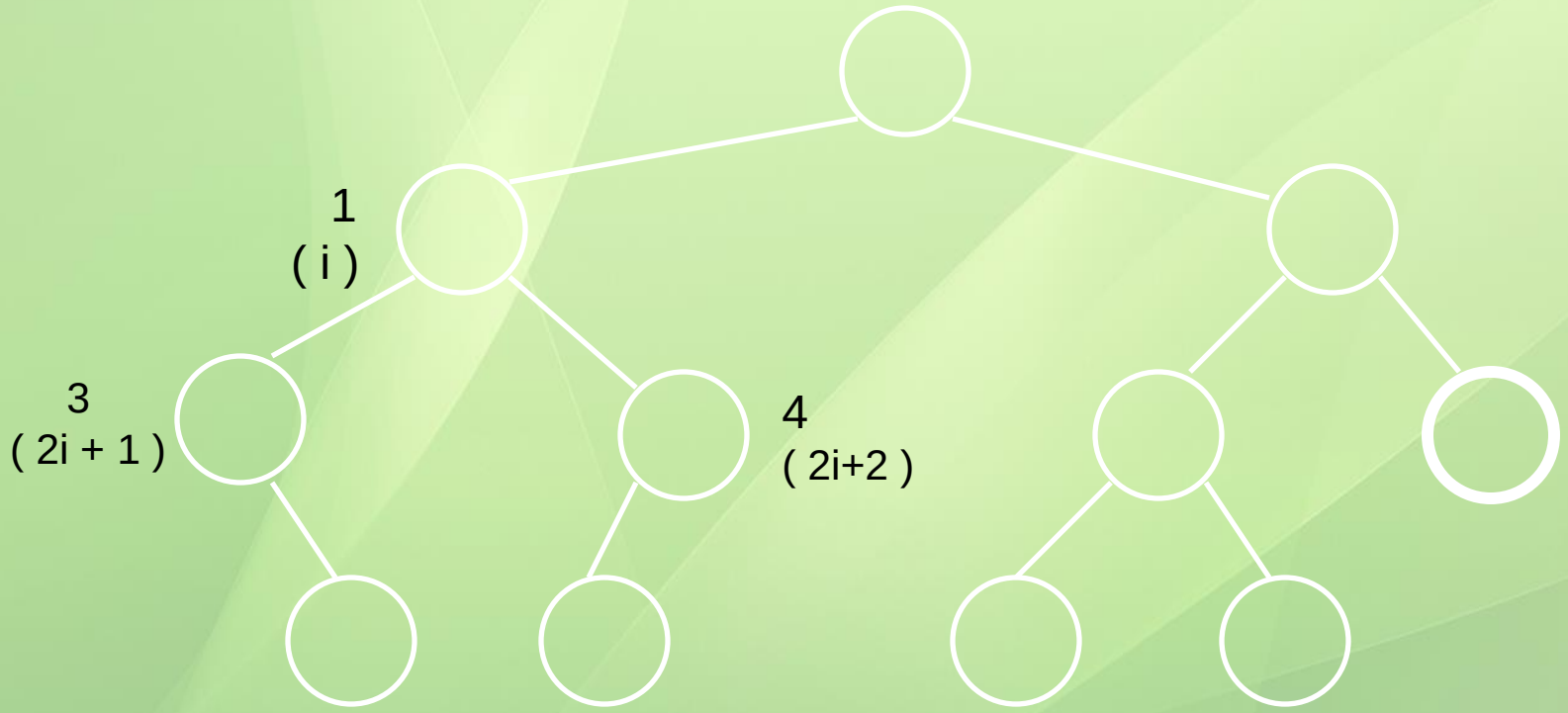
If we consider the root's **position is 0**,

Parent's position(index) = i,

Left child's position(index) = 2i+1,

Right child's position(index) = 2i+2.

Parent Child Relationship



Binary Tree in Array

Child parent relationship (from child to parent):

Child's position (index): k

Parent's position: (k-1)/2,

Where / denotes Integer division.

Child Parent Relationship



Linked representation of Binary Tree

```
struct node
   {
   int data;
   node *lchild;   //pointer to left child
   node *rchild;   //pointer to right child
    };
```

If the tree is not complete then linked representation is efficient.

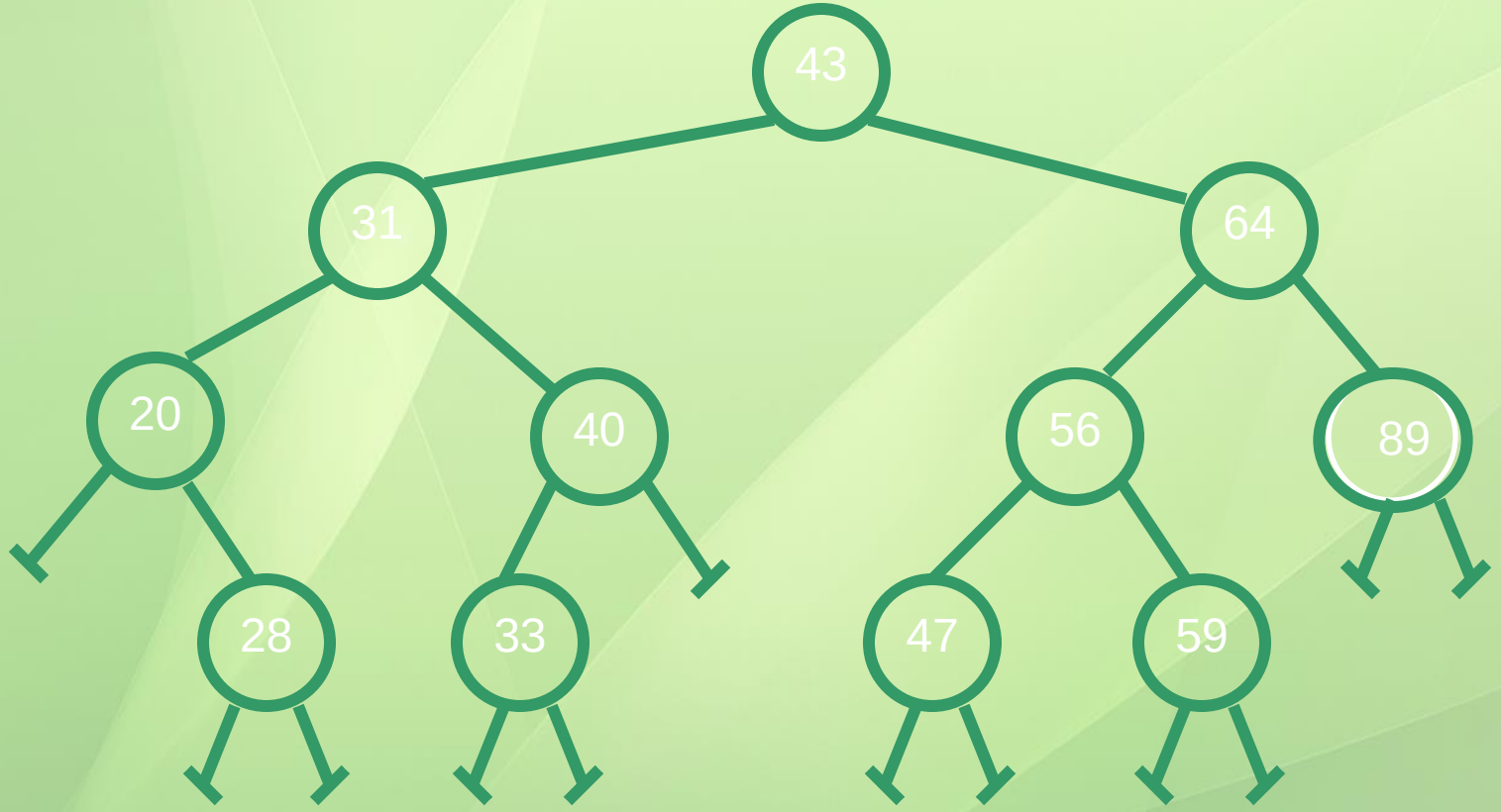
Traversal Technique of a Binary Tree

- There are three main traversal techniques (methods) for a binary tree. Such as
  - Pre-order Traversal Method
  - In-order Traversal Method
  - Post-order Traversal Method

Pre-order Traversal Method

- Visit the root (node).
- Traverse the left sub-tree (in pre-order)
- Traverse the right sub-tree (in pre-order)

Example: Preorder



43	31	20	28	40	33	64	56	47	59	89

Another Example of Preorder

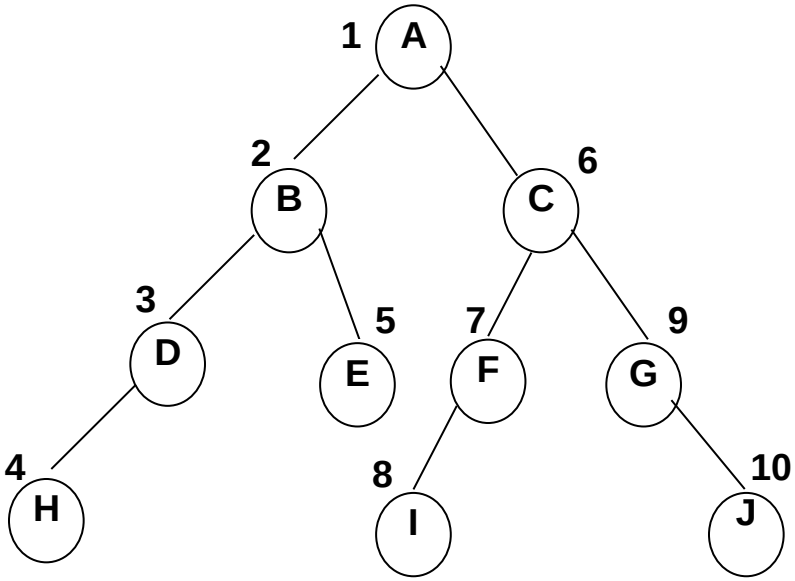


Figure 7.5: Pre-order traversal method

Visiting sequence:

A B D H E C F I G J

Pre-order Traversal Method

Algorithm 7.1: Algorithm for pre-order traversal

1. Input a binary tree
2. preorder (node * curptr)

```
{
   if (curptr!= NULL)
      {
       print curptr→data;
       preorder (curptr→lchild);
       preorder (curptr→ rchild);
       }
}
```

3. Output: the information of the nodes.

Preorder method using array

Array based Algorithm

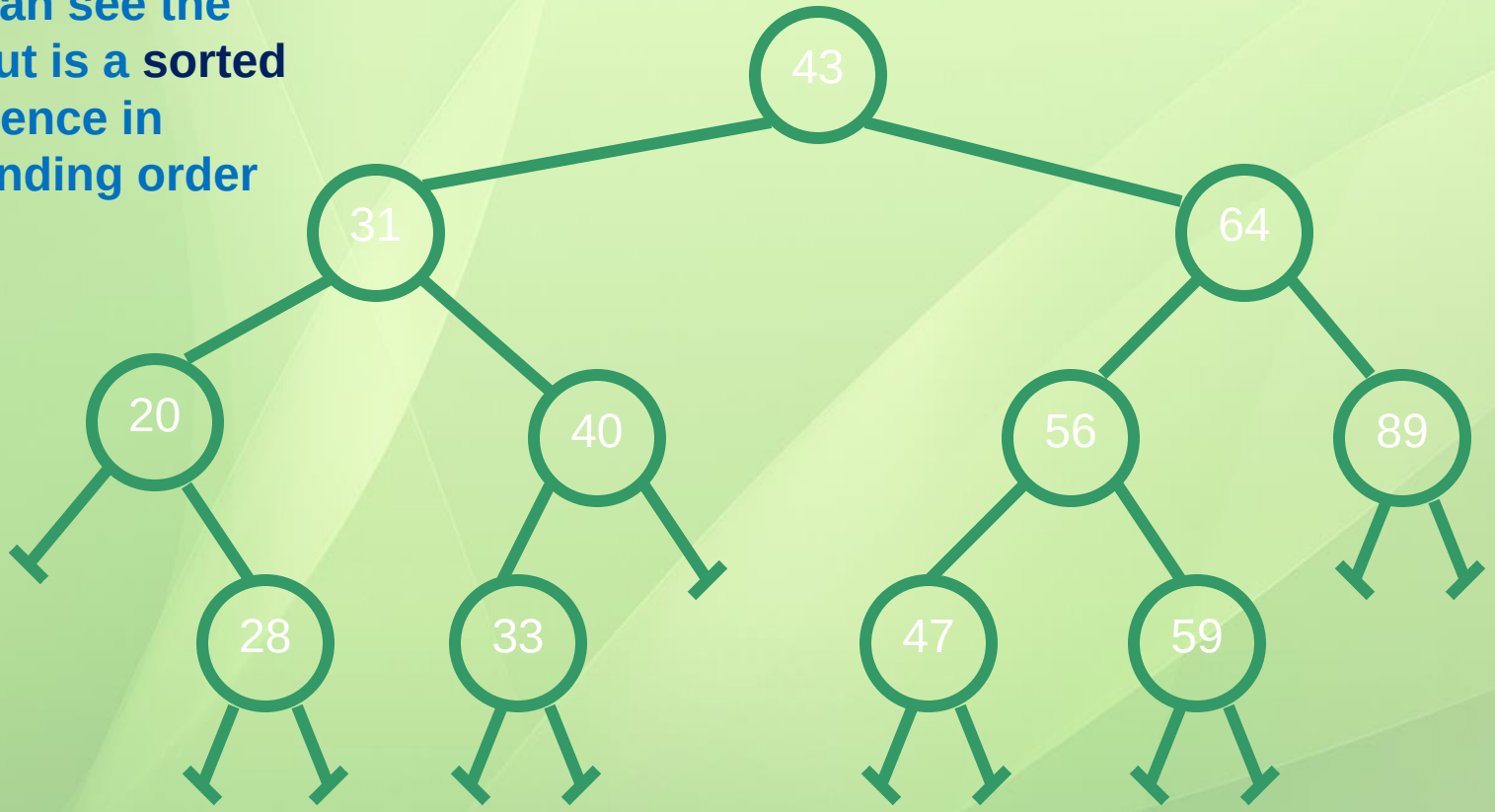
```
preorder (int i, int tree[])
{ if (i<n)
{ print (tree[i]);
l=2*i+1;
preorder(l, tree) // l is the index of left child
r=2*i+2;
preorder(r, tree) //r is the index of right child
main()
{input tree[n];
preorder(0,tree);}
```

Inorder Traversal

- Traverse the left subtree (inorder).
- Visit the root (node).
- Traverse the right subtree (inorder).

Example: Inorder

We can see the output is a sorted sequence in ascending order



20	28	31	33	40	43	47	56	59	64	89

In-order Traversal

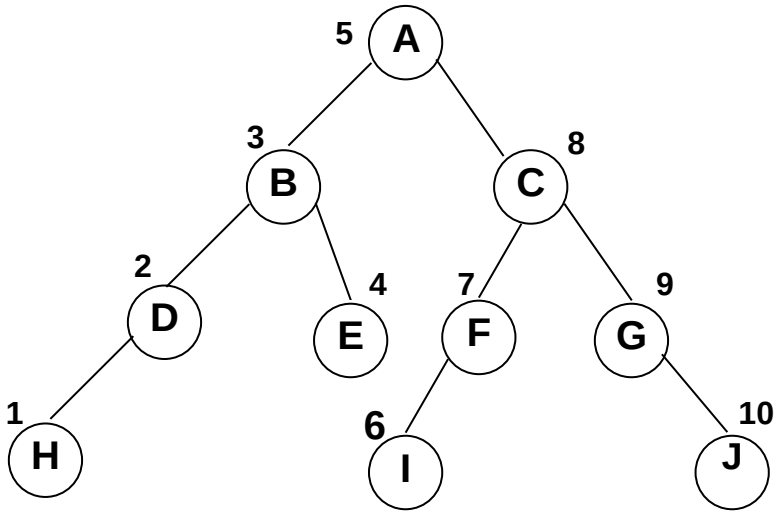


Fig-(a)

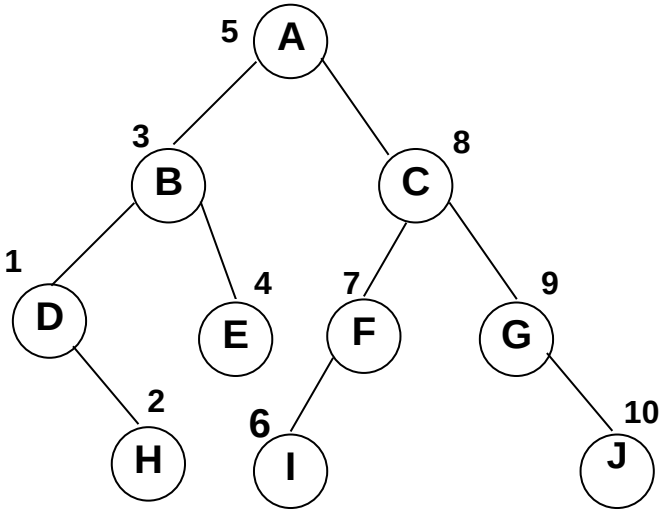


Fig-(b)

Visiting Sequences:

H D B E A I F C G J

D H B E A I F C G J

In-order Traversal Method

Algorithm 7.2: Algorithm for in-order traversal

1. Input a binary tree.
2. inorder (node * curptr)

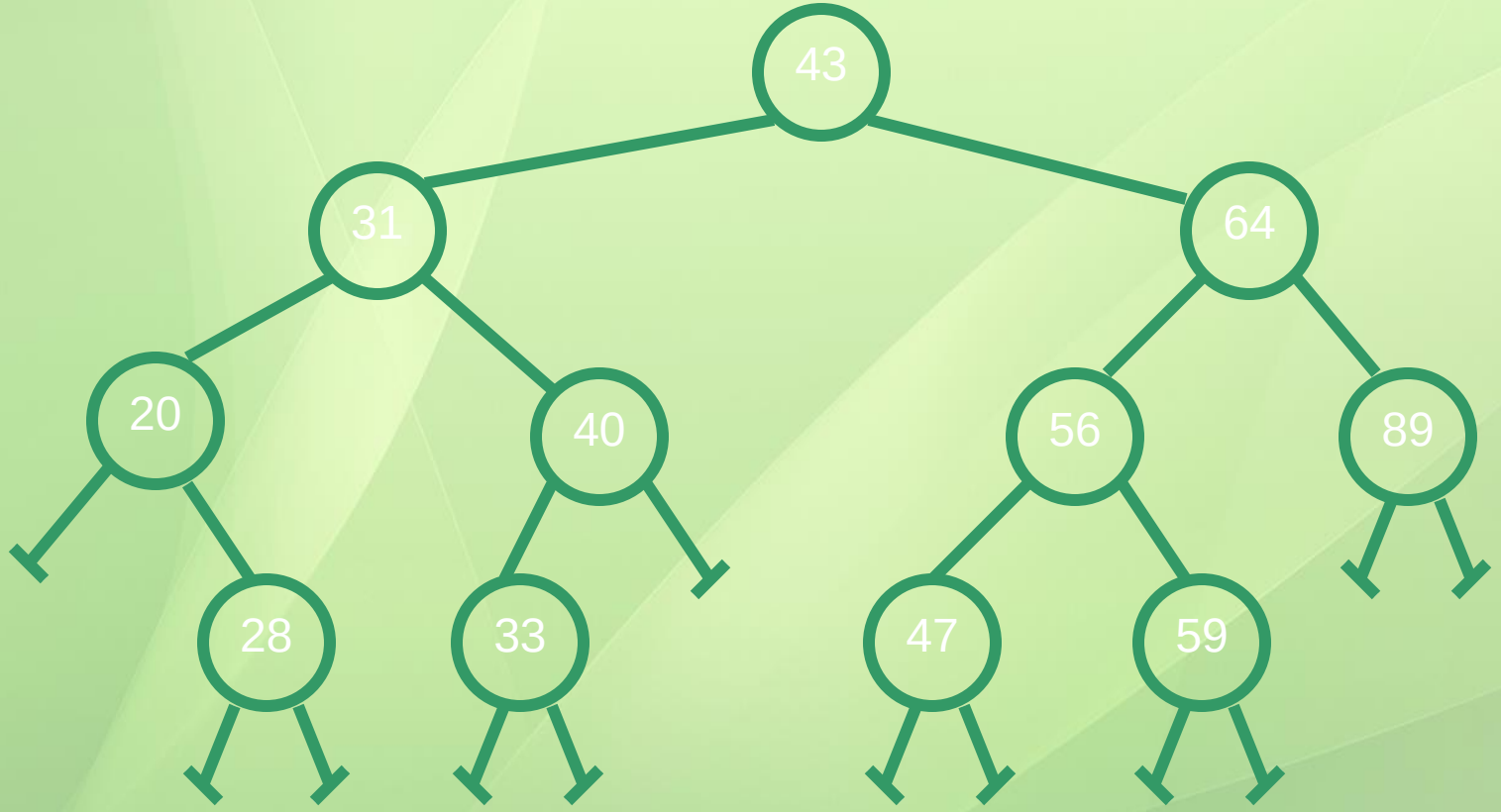
```
{
    if (curptr!= NULL)
    {
        inorder (curptr→lchild);
        print curptr→data;
        inorder (curptr→rchild);
    }
}
```

3. Output: the information of the nodes.

Postorder Traversal

- Traverse the left subtree (postorder).
- Traverse the right subtree (postorder).
- Visit the root (node).

Example: Postorder



28	20	33	40	31	47	59	56	89	64	43

Post-order Traversal Method

Algorithm 7.3: Algorithm for post-order traversal

1. Input a binary tree.
2. postorder (node * curptr)

```
{
if (curptr!= NULL)
        {
        postorder (curptr→lchild);
        postorder (curptr→rchild);
        print curptr→data;
        }
}
```

3. Output: the information of the nodes.

Expression Tree

- A Binary Tree built with operands and operators.
- Also known as a parse tree.
- Use in compilers.

Example: Expression Tree



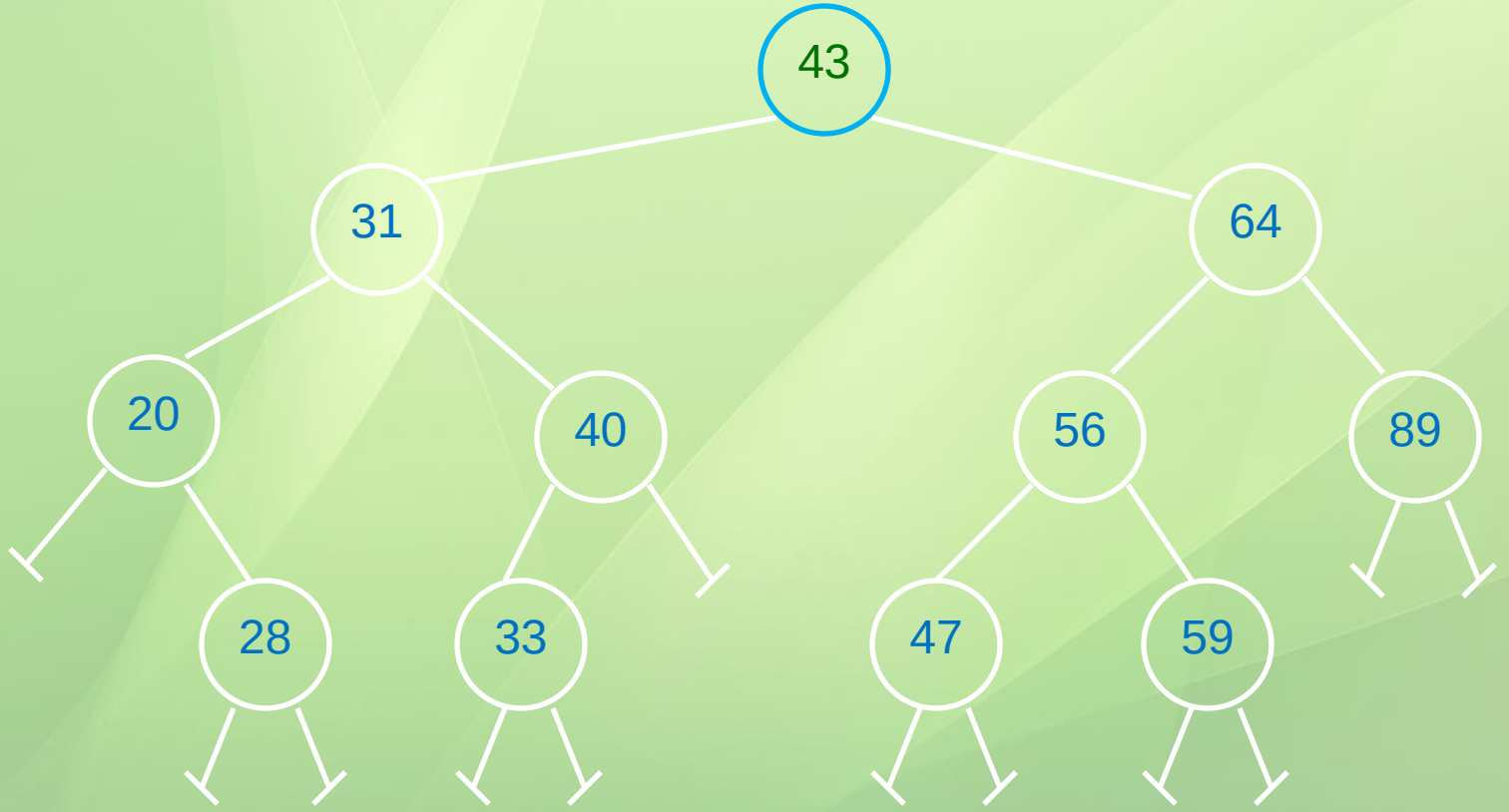
1/3 + 6*7 / 4

Binary Search Tree(BST)

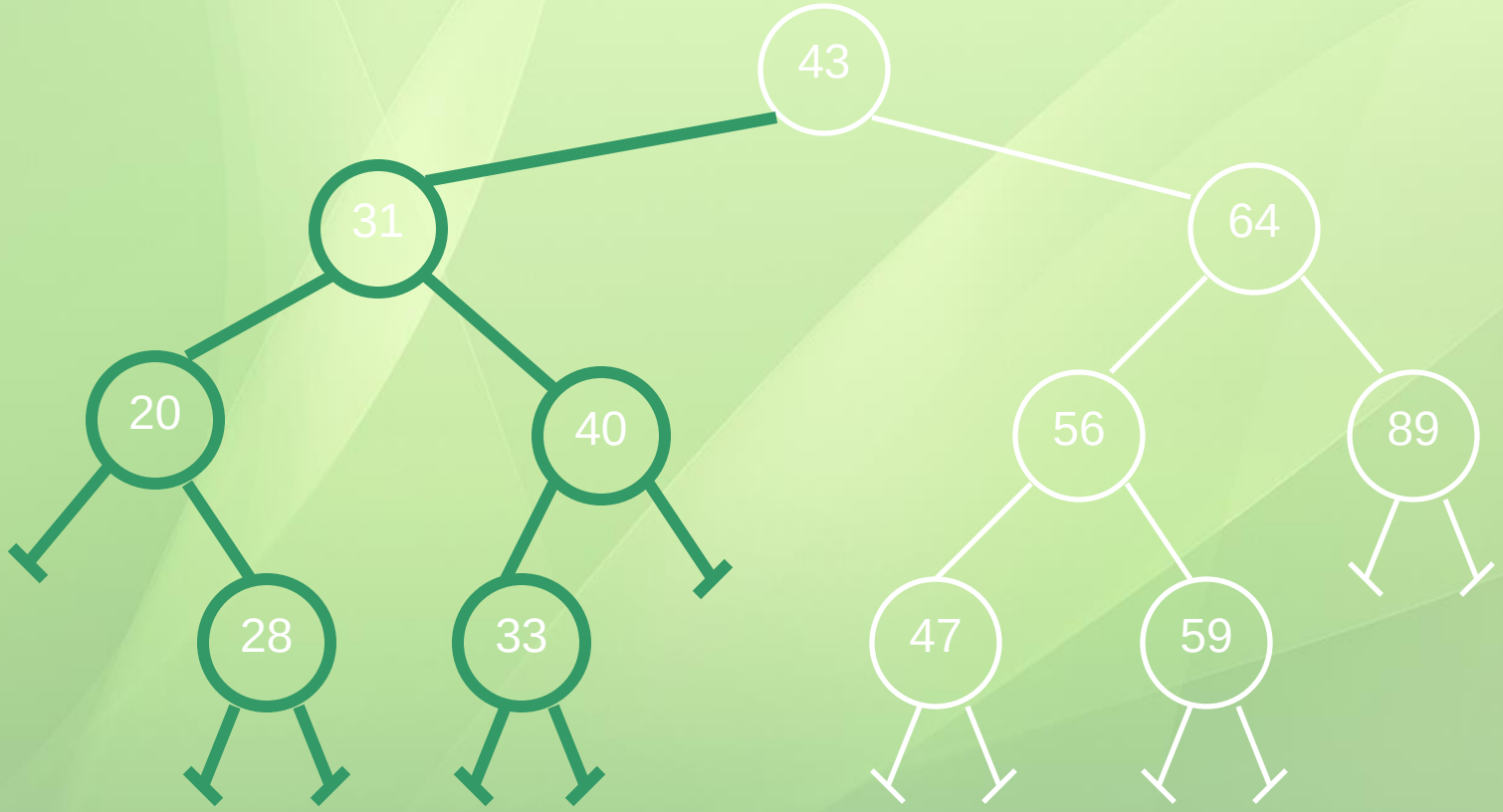
A BST is a Binary Tree such that:

- Every node entry has a unique key or value.
- All the values in the left subtree of a node are smaller(less) than the value of the node.
- All the values in the right subtree of a node are greater than the value of the node.
- Left and rigt sub-trees are also BST.

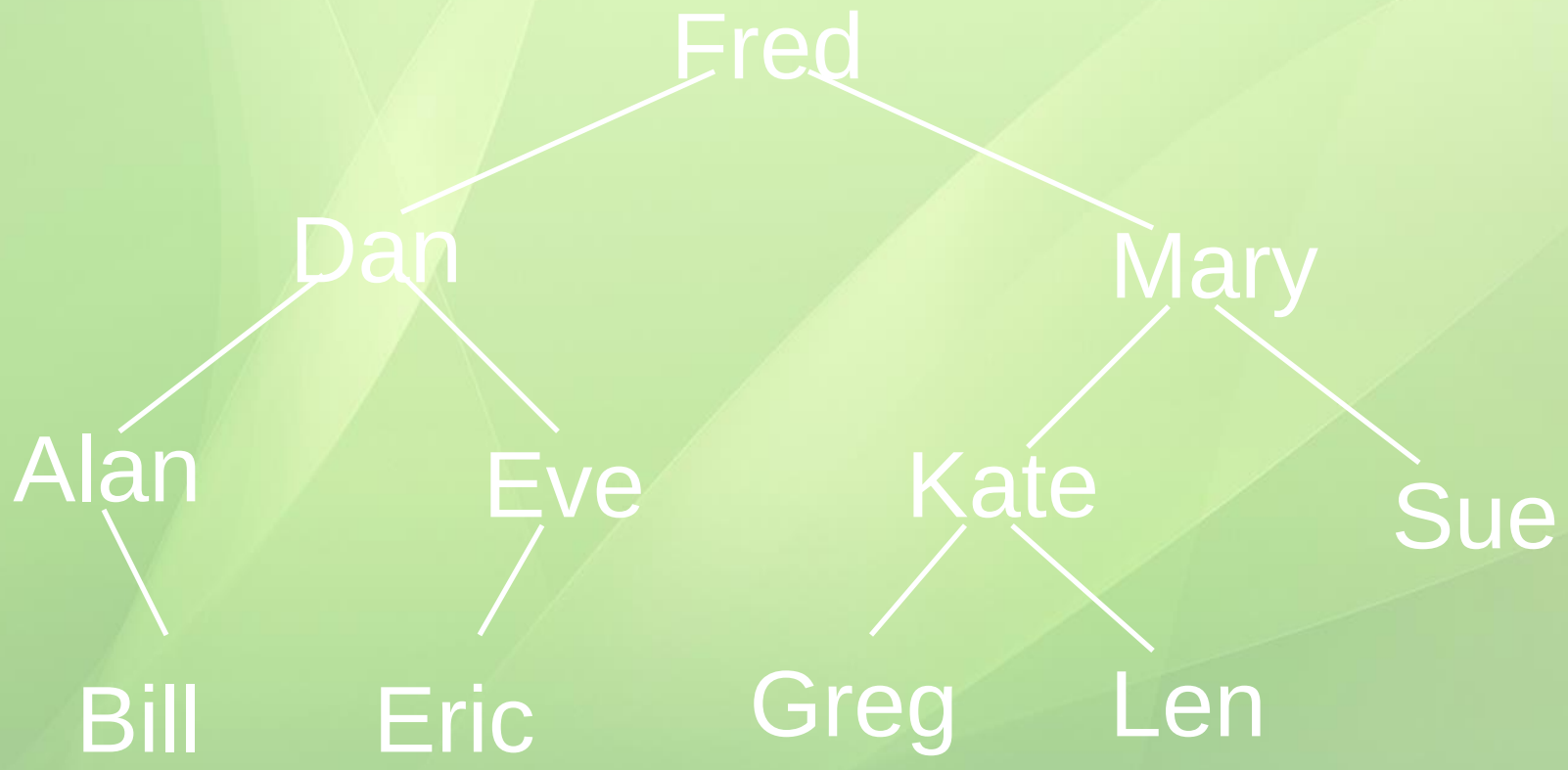
Example 1: *Key or value is an integer*



Example 1: *key or value is an integer*



Example 2: *Key or value is a string*

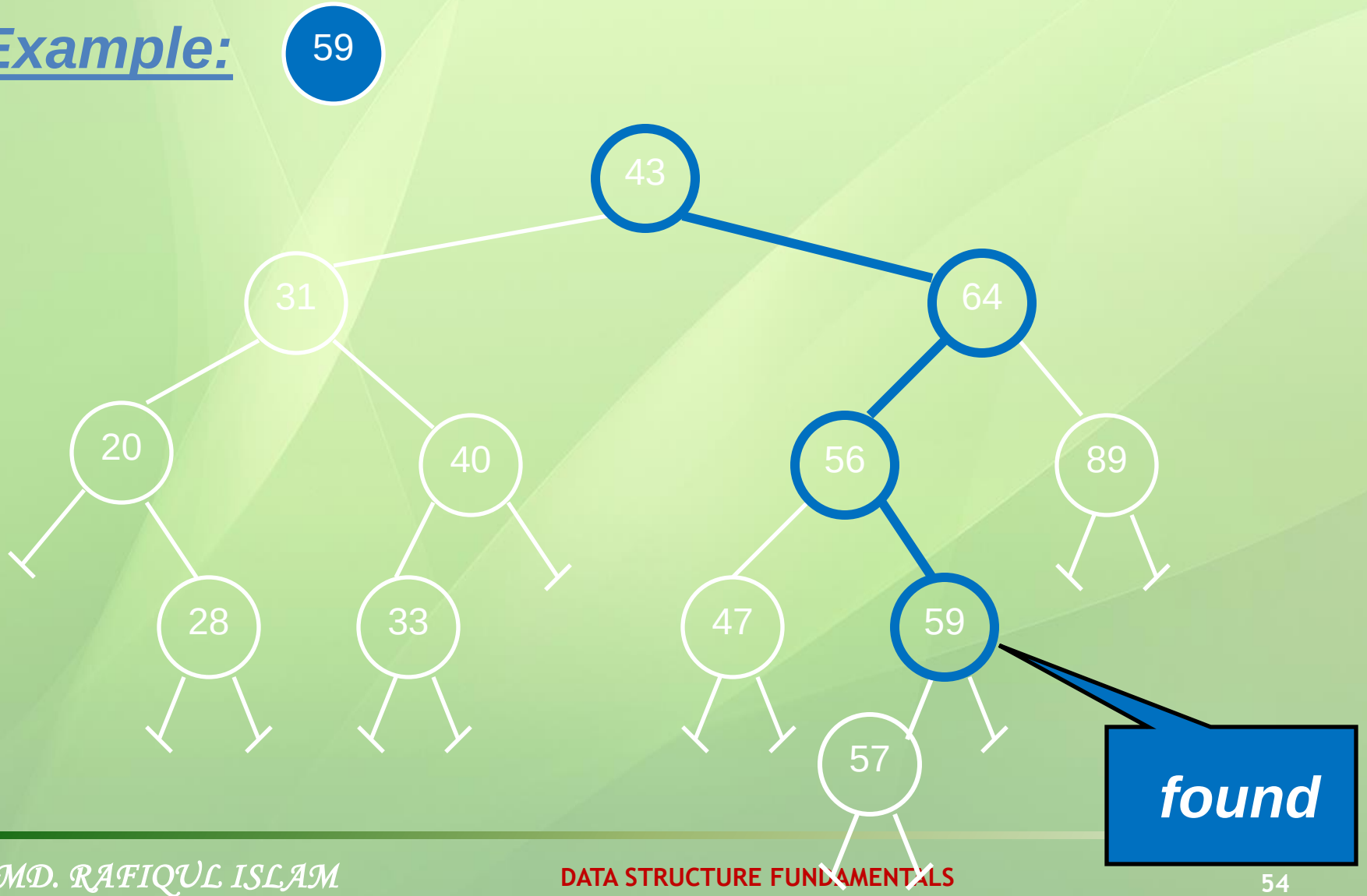


Searching a particular node

- Compare with the root if equal then found.
- Else if the node value is less search the left subtree.
- Otherwise (for greater) search the right subtree.
- Searching cost is $O(\log_2 n)$ if BST is a complete.

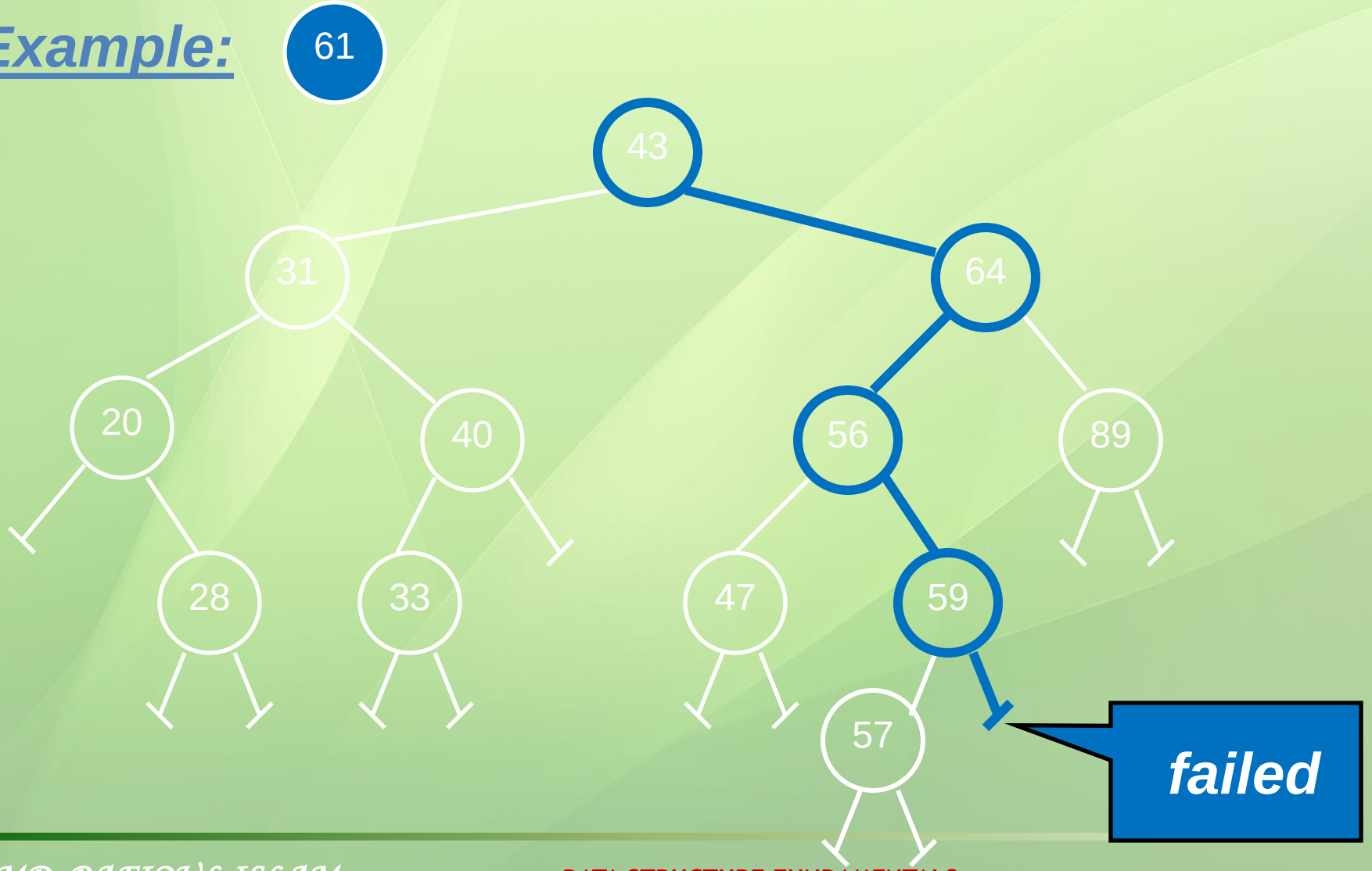
Searching

Example:



Searching

Example:



Searching Algorithm

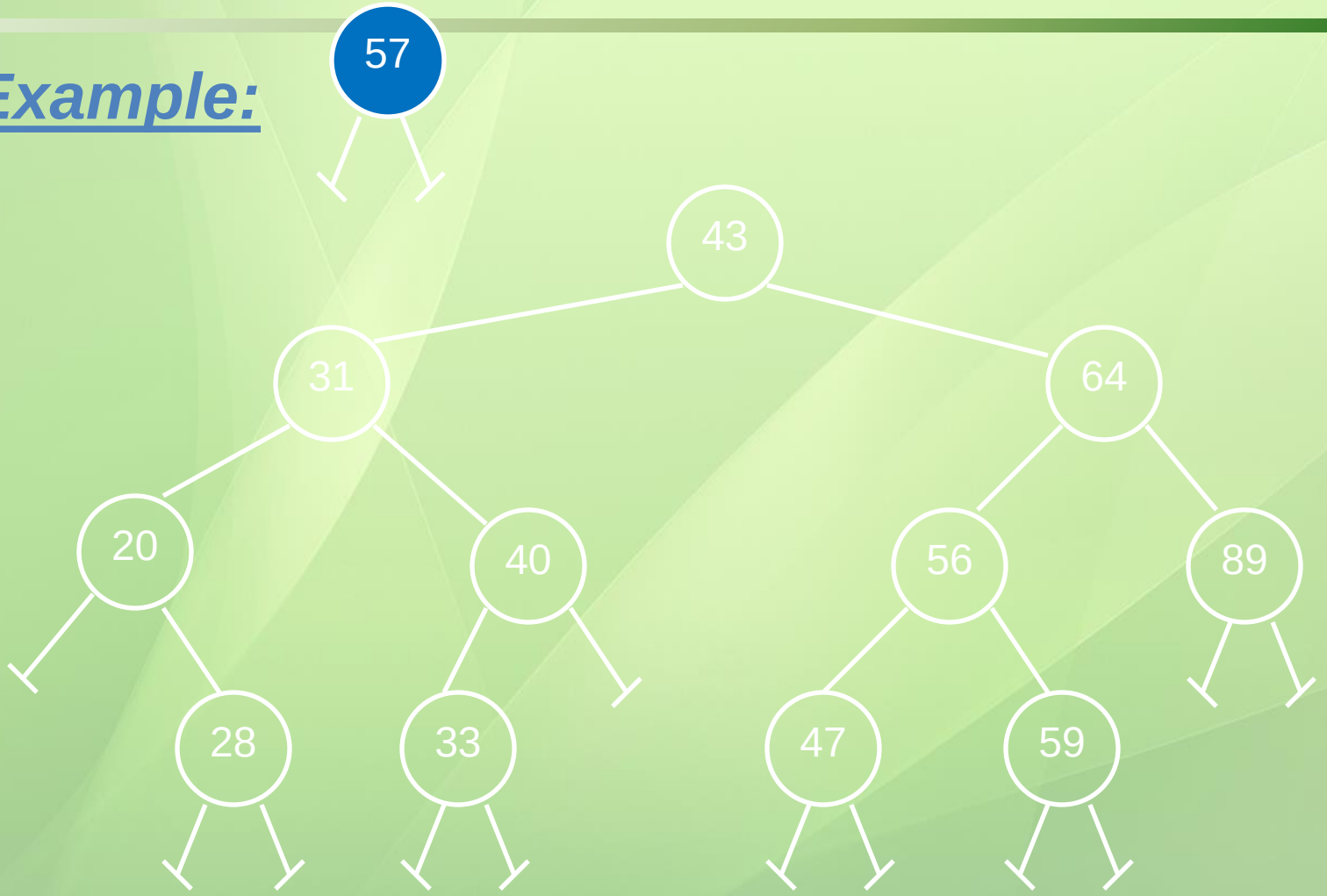
- **Algorithm 7.4:** Algorithm to find out a particular node value of BST
  1. Input BST and a node value, *x*;
  2. Repeat Step-3 to Step-5 until we find the value or we go beyond the tree.
  3. If *x* is equal to root node value, searching is successful (print "Found") and terminate the algorithm.
  4. If *x* is less than root node value, we have to search the left sub-tree (by treating it as a BST).
  5. Else we have to search right sub-tree (by treating it as a BST).
  6. Otherwise the node value is not present in BST (print "Not Found").
  7. Output: Print message "FOUND" or "NOT FOUND"

Insert or Add

- Create new node for the item.
- Find the parent node the node.
- Add or Attach new node as a leaf.

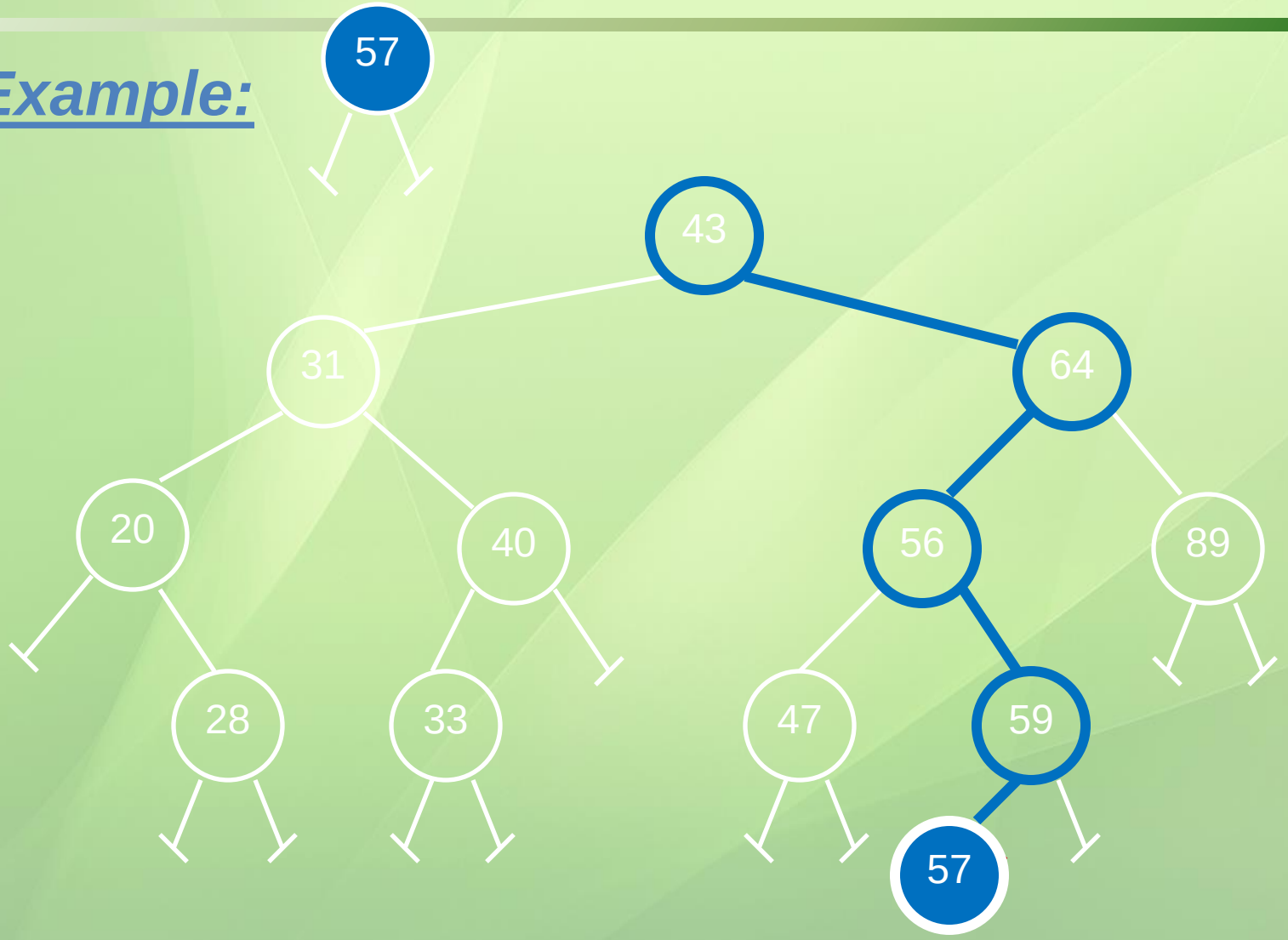
Insert or Add

Example:



Insert or Add

Example:



Algorithm to Add

- **Algorithm 7.5:** Algorithm to insert (add) a node to a BST

1. Input BST and a new node;
2. Repeat Step-3 to Step-5 until we find the value or we go beyond the tree.
3. If x is equal to root node value, searching is successful and terminate the algorithm.
4. If *x* is less than root node value, we have to search the left sub-tree (by treating it as a BST).
5. Else we have to search right sub-tree (by treating it as a BST).
6. Make link between the new node and the parent node of the new node.
7. Output: Updated BST.

Delete a node from BST

- Find or locate the node (searching)

Case 1:

- if the target node is a leaf node, then we just delete the node.

Case 2:

- if the target node has only one child, then we make link between the child and parent node of the target node and delete the node.

Deletion from BST

Case 3:

if the target node has two children (with grand children also).

Search left sub-tree of the target node and find the node with maximum value and mark it.

Replace the node value by the maximum value.

Delete the marked node (the node with maximum value).

(We can do it using right subtree also in thet case minimum value will be used).

Deletion from BST

Example of Case 3:



Deletion from BST

Example of case 3:



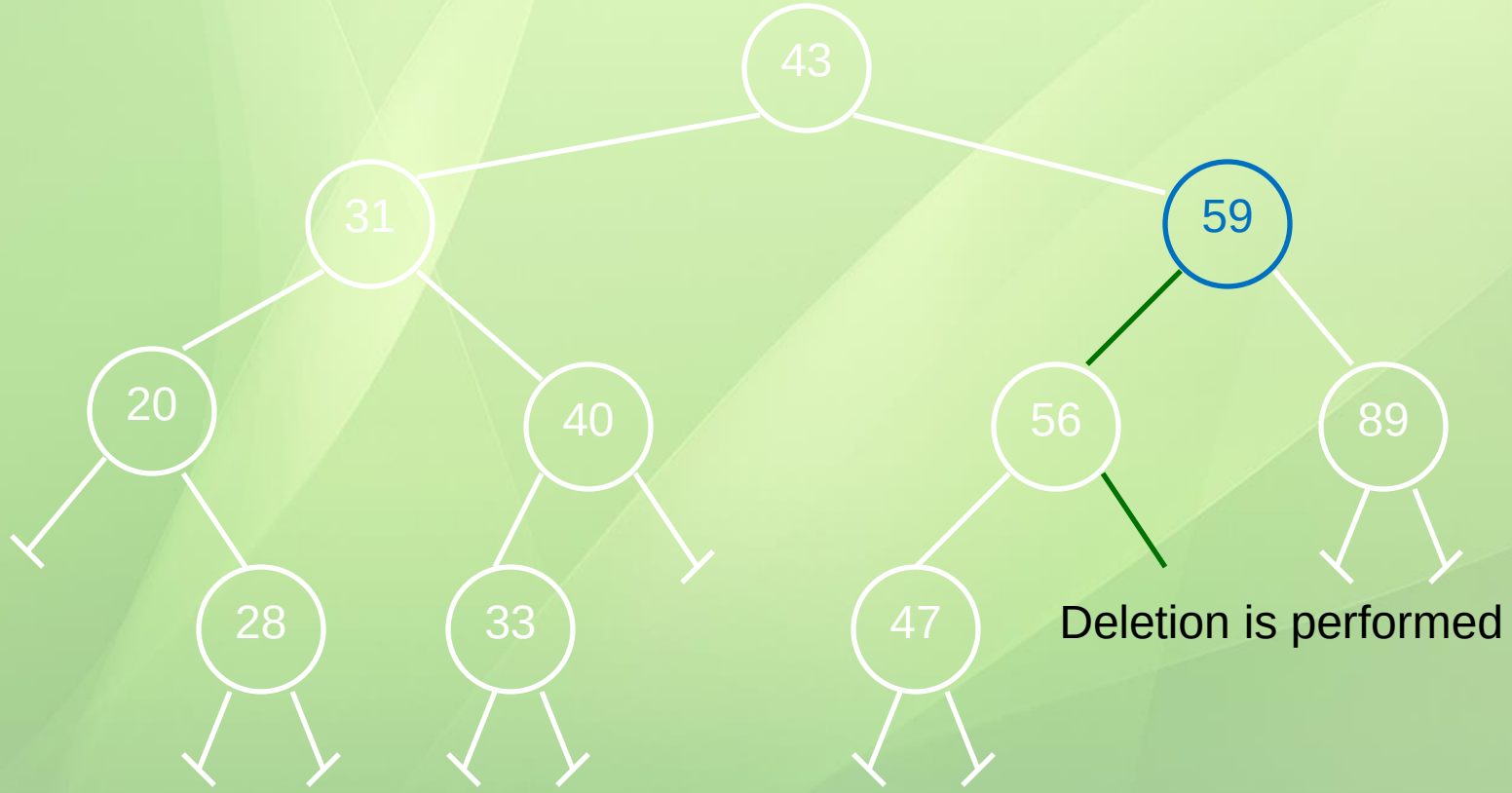
Deletion from BST

Example of case 3:



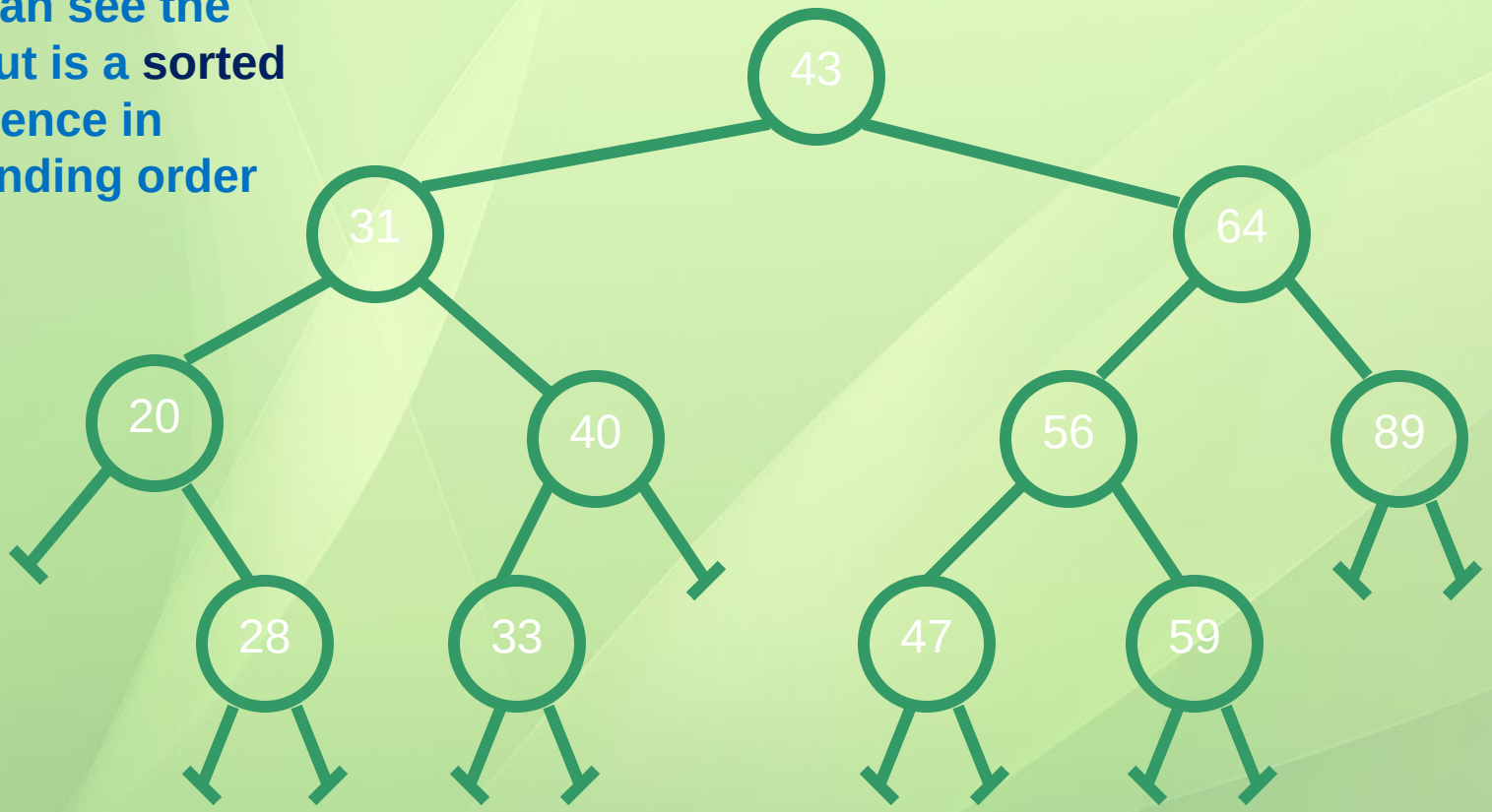
Deletion from BST

Example of Case 3:



Inorder Traversal of a BST

We can see the output is a sorted sequence in ascending order



20	28	31	33	40	43	47	56	59	64	89

Next version of tree

is

Heap